

Classificação Utilizando Convolutional Neural Network

Trabalho II - Tópicos em Aprendizado de Máquina

Ulisses Curvello Ferreira — GRR: 20223829

Pedro Henrique Gurski de Oliveira — GRR: 20224759

Universidade Federal do Paraná

Maio 2025

Sumário

1	Introdução	3
2	Descrição dos Datasets	3
3	Modelos CNN Implementados	4
3.1	SimpleCNN	4
3.2	LightCNN	5
3.3	MiniMobileNetV2	5
4	Resultados obtidos	6
5	Melhorias	7
6	Conclusão	7

1 Introdução

A classificação de imagens é um problema muito amplo na computação, com aplicações que vão desde organização automatizada de fotos até sistemas de assistentes virtuais. Neste trabalho, o objetivo foi desenvolver um sistema capaz de classificar imagens. Para isso, utilizamos dataset público. Escolhemos o **“Shoe vs Sandal vs Boot Dataset”** devido à sua diversidade visual e tamanho razoável para um treinamento adequado de aprendizado profundo. Seu dataset possui cerca de 15 mil imagens. As imagens possuem diferentes condições de iluminação, o que representa um desafio para modelos de visão computacional, tendo como principal tarefa a classificação em três tipos distintos: tênis, sandália e botas.

2 Descrição dos Datasets

O conjunto de dados utilizado neste projeto é o Shoe vs Sandal vs Boot Dataset, disponível de forma fácil no link abaixo através do site kaggle. Foi criado como o objetivo de treinamento, assim pode classificar suas imagens em três diferentes grupos de calçados: botas (Boot), sandálias(Sandal) e sapatos (Shoe).

O dataset contém aproximadamente 15 mil imagens coloridas, balanceadas, divididas em cada uma das três classes mencionadas. Suas imagens estão no formato .jpg e com variações de cores, luminosidade e fundo estão presentes no conjunto, tornando um dataset muito desafiador e bom para treinamentos de classificação automática.

A estrutura do diretório segue um formato simples e intuitivo:

Estrutura do Dataset

```
Shoe vs Sandal vs Boot Dataset/  
|-- Boot/  
|-- Sandal/  
|-- Shoe/
```

Link Dataset: <https://www.kaggle.com/datasets/hasibalmuzdadid/shoe-vs-sandal-vs-boot-dataset-15k-images>

3 Modelos CNN Implementados

Os modelos adotados a seguir representam diferentes abordagens e arquiteturas para a tarefa de classificar imagens. A SimpleCNN adota uma estrutura progressiva com camadas aplicando convoluções e uso de pooling. A lightCNN faz redução da complexidade ao usar pooling adaptativo, tornando o modelo mais leve. O último modelo implementado foi o MiniMobileNetV2, que segue uma linha de maior eficiência, fazendo o uso de blocos residuais. Cada modelo será melhor explorado a seguir.

3.1 SimpleCNN

A SimpleCNN é uma rede convolucional básica, simples de entender e ideal para classificar imagens. Sua estrutura tem três blocos de extração de características, onde cada bloco aplica uma camada de convolução, seguida por uma função de ativação ReLU e uma camada de redução (Max Pooling). A cada bloco, a rede aumenta a quantidade de filtros, utilizando 16, 32 e por fim 64, o que permite captar padrões antes não detectados entre as imagens e reconhecer melhor os padrões.

Após essa extração de características, os dados passam por uma camada de descoberta (Flatten), transformando os mapas de característica em um vetor. Este vetor por sua vez alimentará o classificador, composto por duas conexões, sendo a primeira com 128 neurônios e ativação, segunda camada de Dropout com taxa de 0.5, que ajuda a evitar o overfitting. A segunda camada produzirá a saída final com três valores, ou seja, um valor para cada tipo possível de classificação. Bota, sapato ou sandália.

Um ponto importante que podemos citar aqui é que a função CrossEntropyLoss já faz o cálculo de Softmax, dado às camadas intermediárias, assim melhora a estabilidade do treinamento. Outro ponto é que a entrada da primeira camada linear é calculada automaticamente com base no tamanho da imagem, garantindo que o modelo seja flexível para diferenciar resoluções.

3.2 LightCNN

A LightCNN é uma rede um pouco mais leve que a simples Cnn, dado que usa menos parâmetros em sua última camada na parte final, que faz classificação, assim a simples cnn, usa três camadas de convolução de ativação ReLU, as duas primeiras também usam camadas de redução de tamanho MaxPool2d (reduzem as dimensões da imagem).

O que realmente muda está na terceira camada, esta, por sua vez, usa a AdaptiveAvgPool2d((1, 1)) que resume cada mapa de características em apenas um valor, não importa o tamanho da entrada, ele gerará um vetor de tamanho fixo como 64 valores.

Desse modo, a parede classificatória fica bem mais simples e suscetível a errar na hora de classificar, tento apenas uma camada linear que liga diretamente esses 64 valores às possíveis classes (bota, sapato ou sandália). Isso faz com que a rede tenha menos parâmetros, porém treinará mais rápido e em alguns casos terá menos chances de sofrer um overfitting.

3.3 MiniMobileNetV2

A miniMOBILENetV2 é uma versão mais simplificada do MobileNetV2, uma arquitetura conhecida por ser leve e rápida, ideal para dispositivos móveis ou com pouco recurso. Sua principal diferença são os blocos residuais invertidos, que usam convolução separável em profundidade. Uma técnica que reduz bastante o número de cálculos.

O modelo inicia com uma convolução normal e uma normalização dos dados Batch-Norm. Depois, uma sequência de blocos InvertedResidual. Cada bloco com três partes: primeira aumenta os canais, segunda uma convolução separável em profundidade e terceira uma que reduz os canais (projeção). Cada camada é seguida por uma função de ativação ReLU6, melhorando o fluxo de informação.

Ao fim dos blocos, uma última convolução 1x1 aumenta os canais antes de aplicar o AdaptiveAvgPool2d(1) que gera um vetor com informações importantes. Esse vetor por sua vez é achatado (Flatten) e passa por uma camada de Dropout uma linear, que gera valores finais para a classificação, três valores, uma para cada classe, assim buscando um bom equilíbrio entre desempenho e leveza.

4 Resultados obtidos

A SimpleCNN apresentou resultados consistentes ao longo dos folds da validação cruzada. A Acurácias médias girou em torno de 94% e 95%, com métricas de precisão, recall e f1 score igualmente elevado. Todas muito próximas e como baixa variação, indicando que o modelo consegue obter bom resultado e tem boa capacidade e bom equilíbrio e eficiência.

A LightCNN foi a que teve o pior desempenho em questão, dado os folders. As suas métricas como acurácia, previsão e f1 score ficaram na faixa de 64 e 70 . Evidenciando uma limitação na capacidade de classificação do modelo. Apesar da proposta de ser leve, o uso de número reduzido de parâmetros e como apenas uma camada linear após o pooling adaptativo, pode ter comprometido sua habilidade de discriminar adequadamente entre as classes.

O modelo MiniMobileNetV2 apresentou uma grande oscilação entre os folds, enquanto em alguns obteve acurácias acima de 90%, em outros obteve valores abaixo de 70%. Isso sugere que o modelo é potencialmente poderoso, mas sua eficácia depende bastante da qualidade e equilíbrio do subconjunto. A arquitetura, baseada em blocos residuais invertidos e convoluções separáveis em profundidade, é bastante eficiente em termos computacionais, mas pode exigir mais dados. Acredito que esse tenha sido o principal problema do modelo.

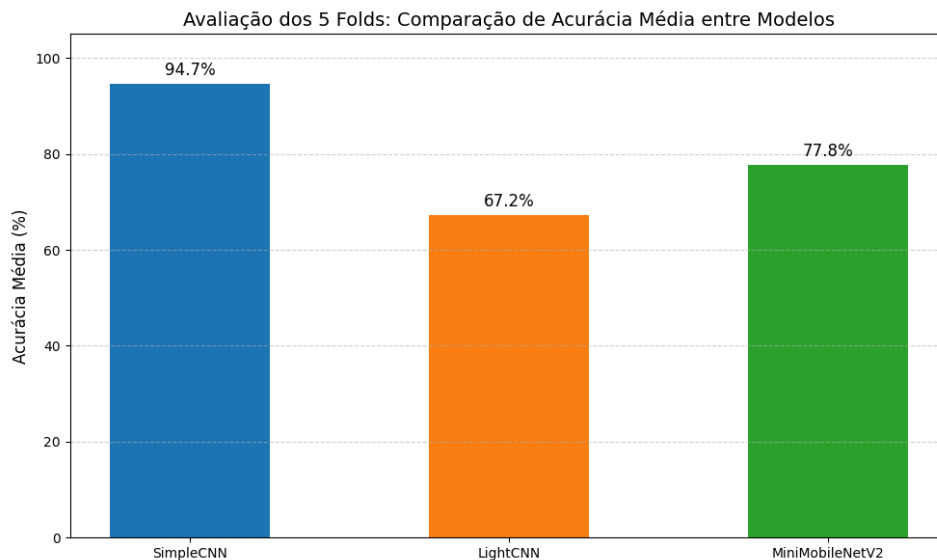


Figura 1: Desempenho dos modelos nos diferentes folds de validação cruzada

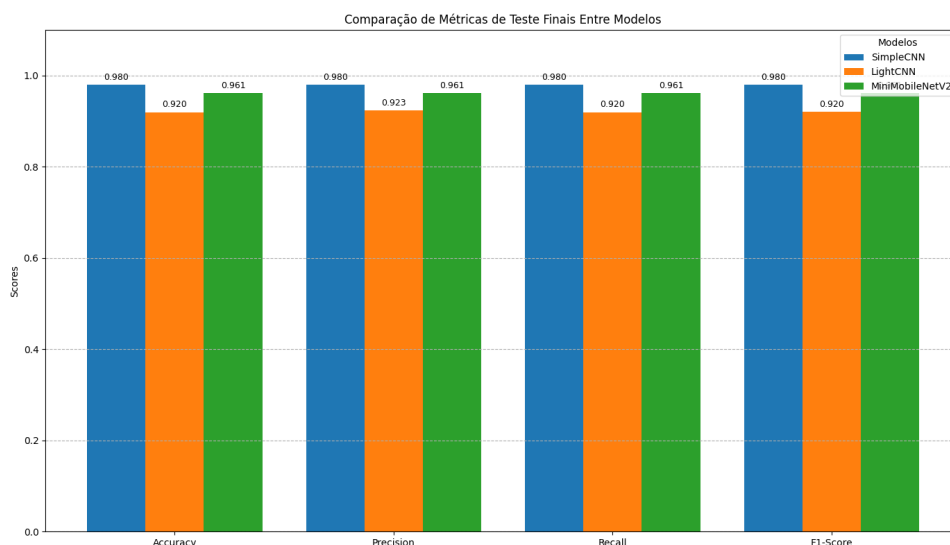


Figura 2: Desempenho dos modelos nos diferentes folds de validação cruzada

Resultados dos modelos com 20 épocas de treinamento

5 Melhorias

Explorar diferentes combinações como taxa de aprendizado, otimizador, tamanho e técnicas de regularização para encontrar a configuração que oferece o melhor desempenho para cada modelo, assim aumentando a sua eficiência. Utilizar técnicas para aumento de dados durante o treinamento com o objetivo de tornar os modelos mais robustos e capazes de generalizar melhor, especialmente quando pode haver overfitting.

Testar modelos de CNN mais complexos ou reaproveitar redes já treinadas em conjuntos grandes, fazendo uma adaptação. Isso pode melhorar significativamente o desempenho. Junto a isso estender o tempo de treinamento ao aumentar o número de épocas e acompanhando a perda para evitar possível overfitting.

Por fim, examinar onde porque os modelos erram, a fim de entender as causas desses erros e buscar melhorias no modelo.

6 Conclusão

Neste trabalho, exploramos diferentes arquiteturas de redes neurais convolucionais para a tarefa de classificação do dataset “Shoe vs Sandal vs Boot”. Os modelos implementados demonstraram comportamentos distintos, refletindo o equilíbrio entre complexidade, capacidade de generalização e eficiência computacional. Como trabalhos futuros, destacamos a exploração de técnicas de aumento de dados, ajustes de hiperparâmetros e o uso de arquiteturas pré-treinadas, visando a melhoria da precisão e robustez dos modelos.